

NAME

imarith -- binary image arithmetic

USAGE

imarith operand1 op operand2 result

PARAMETERS**operand1,operand2**

Lists of images and constants to be used as operands. Image templates and image sections are allowed.

op Operator to be applied to the operands. The allowed operators are "+", "-", "*", "/", "min", and "max".

result

List of resultant images.

title = ""

Title for the resultant images. If null("") then the title is from operand1 if operand1 is an image or from operand2 otherwise.

divzero = 0.

Replacement value for division by zero. When the denominator is zero or nearly zero the result is replaced by this value.

hparams = ""

List of header parameters to be operated upon. This is primarily used for adding exposure times when adding images.

pixtype = "", calctype = ""

Pixel datatype for the resultant image and the internal calculation datatype. The choices are given below. They may be abbreviated to one character.

""

Calctype defaults to the highest precedence operand datatype. If highest precedence datatype is an integer type and the operation is division then the calculation type will be "real". If the highest precedence operand is type "ushort", *calctype* will default to "long". *Pixtype* defaults to *calctype*. Users who want type "ushort" images on output will need to set *pixtype* to "ushort" explicitly.

"1", "2"

The pixel datatype of the first or second operand.

"short", "ushort", "integer", "long", "real", "double"

Allowed IRAF pixel datatypes.

verbose = no

Print the operator, operands, calculation datatype, and the resultant image name, title, and pixel datatype.

noact = no

Like the verbose option but the operations are not actually performed.

DESCRIPTION

Binary image arithmetic is performed of the form:

operand1 op operand2 = result

where the operators are addition, subtraction, multiplication, division, and minimum and maximum. The division operator checks for nearly zero denominators and replaces the ratio by the value specified by the parameter *divzero*. The operands are lists of images and numerical constants and the result is a list of images. The number of elements in an operand list must either be one or equal the number of elements in the resultant list. If the number of elements is one then it is used for each resultant image. If the number is equal to the number of resultant images then the elements in the operand list are matched with the elements in the resultant list. The only limitation on the combination of images and constants in the operand lists is both operands for a given resultant image may not be constants. The resultant images may have the same name as one of the operand images in which case a temporary image is created and after the operation is successfully completed the image to be replaced is overwritten by the temporary image.

If both operands are images the lengths of each axis for the common dimensions must be the same though the dimensions need not be the same. The resultant image header will be a with the greater dimension. If the dimensions are the same then image header for the resultant image is copied from operand1. The title of the resultant image may be changed using the parameter *title*. The pixel datatype for the resultant image may be set using the parameter *pixtype*. If no pixel datatype is specified then the pixel datatype defaults to the calculation datatype given by the parameter *calctype*. The calculation datatype defaults to the highest precedence datatype of the operand images or constants except that a division operation will default to real for integer images. The precedence of the datatypes, highest first, is double, real, long, integer, and short. The datatype of a constant operand is either short integer or real. A real constant has a

Arithmetic on images of unequal dimensions implies that the operation is repeated for each element of the higher dimensions. For example subtracting a two dimensional image from a three dimensional image consists of subtracting the two dimensional image from each band of the three dimensional image. This works for any combination of image dimensions. As an extreme example dividing a by a one dimension image consists of dividing each line of each plane of each band ... by the one dimensional image.

There are two points to emphasize when using images of unequal dimensions. First, a one dimensional image operates on a line of a two or higher dimension image. To apply a to the columns of a higher dimensional image increase the image dimensionality with **imstack**, transpose the resultant image, and then replicate the columns with **blkrep** (see the EXAMPLE section). The second point of confusion is that an image with a size given by **imheader** of [20,1] is a while an image with size of [20] is a one dimensional image. To reduce the dimensionality of an image use **imcopy**.

In addition to operating on the image pixels the image header parameters specified by the list *hparams* are also operated upon. The operation is the same as performed on the pixels and the values are either the values associated with named header parameters or the operand constant values. The primary purpose of this feature is to add exposure times when adding images.

The verbose option is used to record the image arithmetic. The output consists of the operator, the operand image names, the resultant image name and pixel datatype, and the calculation datatype.

EXAMPLES

1. To add two images and the exposure times:

```
cl> imarith ccd1 + ccd2 sum
>>> hparams="itime,otime,ttime,exposure"
```

2. To subtract a constant from an image and replace input image by the subtracted image:

```
cl> imarith m31 - 223.2 m31
```

Note that the final pixel datatype and the calculation datatype will be at least of type real because the constant operand is real.

3. To scale two exposures, divide one by the other, and extract the central portion:

```
cl> imarith exp1[10:90,10:90] * 1.2 temp1
cl> imarith exp2[10:90,10:90] * 0.9 temp2
cl> imarith temp1 / temp2 final title='Ratio of exp1 and exp 2'
cl> imdelete temp1,temp2
```

Note that in this example the images temp1, temp2, and final will be of real pixel datatype (or double if either exp1 or exp2 are of pixel datatype double) because the numerical constants are real numbers.

4. To divide two images of arbitrary pixel datatype using real arithmetic and create a short pixel datatype resultant image:

```
cl> imarith image1 / image2 image3 pixtype=short \
>>> calctype=real title="Ratio of image1 and image2"
```

5. To divide several images by calibration image using the image pixel type of the numerator images to determine the pixel type of the calibrated images and the calculation arithmetic type:

```
cl> imarith image1,image2,image3 / calibration \
>>> image1a,image2a,image3a pixtype=1 calctype=1
```

The same operation can be done in place with image template expansion by:

```
cl> imarith image* / calibration image* pixtype=1 calctype=1
```

6. To subtract a two dimensional bias from stacked observations (multiple two dimensional observations stacked to form a three dimensional image):

```
cl> imarith obs* - bias obs//b
```

Note that the output observations obs101b, ..., will be three dimensional.

7. To divide a 50 x 50 image by the average column:

```
cl> blkavg img avcol 50 1
cl> blkrep avcol avcol 50 1
cl> imarith img / avcol flat
```

8. To subtract a one dimensional image from the lines of a image:

```
cl> imarith im2d - im1d diff
```

9. To subtract a one dimensional image from the columns of a image:

```
cl> imstack im1d imcol
cl> imtranspose imcol imcol
cl> blkrep imcol imcol 100 1
cl> imarith im2d - imcol diff
```

Note the need to make a two dimensional image with each column replicated since a one dimensional image will operate on the lines of a two dimensional image.

SEE ALSO

blkrep, imdivide, imfunction, imstack, imtranspose